

alternative-implementations

Alternative runtime implementations that satisfy the same C1

Status: design proposal, not an implementation. Untracked — not part of the build or the acceptance suite. Sits alongside `ARCHITECTURE.md` as a "what else could live inside the C1 boundary" companion.

Date: 2026-08-19. Grounded in the working tree at that point.

Update 2026-08-21 — the baseline these proposals assumed has since been simplified. The engine they port was four-disposition (`WRITTEN` / `EXCLUDED` / `REJECTED` / `DEDUPLICATED`) with a `filter(exclude)` and a `dedup` stage, initial/delta run kinds, and mixed-format TDS. [docs/PLAN-CHANGES-21082026.md](#) has landed, collapsing it to two dispositions (`WRITTEN` / `REJECTED` , `parse` → `map` → `schema`), one full snapshot per `run_id` , and homogeneous TDS. Consequences for this document: the C1 invariant, the harness oracle and the criteria count below are corrected in place to the target; the `filter(exclude)` and `dedup` port-map rows in Proposals B/C/E describe stages the simplification **removes** — under the target they are deleted, not ported, and "four dispositions → four sinks/tables" becomes "two dispositions → two"; the trade-off prose keyed on `dedup` (e.g. Proposal E's keyed-state argument) is moot under the target. The runtime-choice conclusions (which engine) are orthogonal to the simplification and stand.

What this document is

`README.md` and `ARCHITECTURE.md` state the load-bearing rule for this project (`README.md:107-110`, `ARCHITECTURE.md:379-381`):

C1 is the specification, not a picture of this repo. The platform is GCP and stays GCP; what varies is the runtime inside the boundary.

The current runtime inside that boundary is **Apache Beam Python** (DirectRunner locally, DataflowRunner on GCP via Flex Templates), orchestrated by **Cloud Composer / Airflow**, with **Dataform** for SQL transforms.

This document proposes **four** alternative runtime choices, each living inside the same C1 boundary. Three leave C1 byte-identical; one (the CDC/streaming variant) modifies a single input edge and is flagged as a variant rather than a drop-in swap.

No code is changed. This is a read/review/propose artefact.

The C1 invariant — what every proposal preserves

Across all four proposals (with E the documented exception on one edge):

- **C1 System Context diagram** — the 5 actors (Migration Operator, Auditor / Data Owner, Migration Engineer, Platform / SRE Operator, Loader Team), the one software system ("routes every record through the two-door engine, loads survivors into Target System and BigQuery, proves `src_read == migrated + not_migrated`"), and the 7 external systems (Mainframe, Target System, Downstream Consumers, GCP, Source Control, KMS, Monitoring). **Unchanged for A-D; one input edge revised for E.**
- **contracts/** — `artefacts.json` + mapping YAML. The single edit point. Unchanged as the contract of record; how it is *consumed* varies by runtime (imported by Python/Java engine code today; read by a SQL generator in proposal C).
- **The two-door engine** — fixed stage order `parse → filter(exclude) → dedup → map → schema`; four dispositions (`written`, `excluded`, `rejected`, `duplicates`); the balancing equation `src_read = migrated + not_migrated = written + excluded + rejected + duplicates` enforced by `require_balance()`; the six enumerated reject reasons (`PARSE_SHORT_RECORD`, `PARSE_BAD_NUMERIC`, `PARSE_BAD_DATE`, `PARSE_INVALID_JSON`, `MAP_UNMAPPED_ENUM_VALUE`, `SCHEMA_INVALID`).

Superseded 2026-08-21 (see the note at the top). Under the target the stage order is `parse → map → schema`, the dispositions are `written` / `rejected`, and the equation is `src_read = written + rejected`. The `filter(exclude)` and `dedup` stages and the `excluded` / `duplicates` dispositions are removed. The six reject reasons above are all PARSE/MAP/SCHEMA reasons and are unchanged.

- **record_lineage + run_ledger** — per-record naming of every not-migrated record (door / stage / reason / source_key); `run_ledger` = persisted door tallies; recon reads observations and cross-checks ledger vs lineage.
- **The artefact contract** — `.FLG` semaphore, `.CHS` checksums, `.ERR`, `.RPT`, `.DAT`. Loaded from `contracts/artefacts.json` by both the Python and Java sides. The Java extractor / loader / recon-service / target-system-mock stay in every proposal.
- **The harness oracle** — `harness/generate.py` generates 405 synthetic records (400 written + 5 rejected) and a `manifest.json` answer key **before** the pipeline runs; deterministic via `--seed`; supports both copybook and CSV layouts (both produce identical parsed fields). **Runner-agnostic.** This is the fact that makes every proposal valid: the harness proves the *expected* numbers independent of any runtime.
- **The 8 acceptance criteria** (`tests/acceptance.py`) — they assert **BigQuery tables + GCS artefacts + manifest numbers + balance, not the runner**. Criteria assert C1-faithful outcomes, not Beam-faithful mechanics. This single fact is what makes each alternative valid.

What varies

The **runtime inside the boundary**: the orchestrator, the data engine, and (optionally) the SQL-transform tool and the container hosting. Everything above the line is fixed; everything below it is swappable.

Concern	Today (the implementation under review)
Orchestration	Cloud Composer / Airflow (<code>composer/dags/mig_000001_1.py</code>)
Local orchestrator	<code>local/scripts/run_pipeline.py</code> (shells out to each stage in sequence)
Data engine	Apache Beam Python — DirectRunner locally, DataflowRunner on GCP (Flex Templates)
Warehouse	BigQuery
SQL transforms	Dataform SQLX
Object store	GCS
Event bus	Kafka / Redpanda
Java apps (Maven)	extractor-app, loader-app, recon-service, target-system-mock
Contract layer	<code>contracts/</code> (<code>artefacts.json</code> + mapping YAML)

Proposal B — Apache Spark on Dataproc Serverless (batch-native)

Swaps

Beam → PySpark on **Dataproc Serverless** (pay-per-batch, no cluster to manage). Composer stays (`DataprocSubmitPySparkOperator`). Local: `pyspark` local mode. Dataform, the Java apps, BigQuery, GCS, Kafka and Terraform stay.

Engine port map

- `route_intake` / `survivor_rank` → Pandas UDF / `mapInPandas` (shared engine imported unchanged).
- `filter(exclude)` → `WHERE` on `client_type`.
- `dedup` → `window().partitionBy("dedup_key").orderBy(survivor_rank) ; take row_number() == 1` as the survivor, `> 1` as duplicates. The direct distributed equivalent of the Beam `GroupByKey`.
- the four dispositions → four DataFrames tagged by a disposition column → four BigQuery tables via the Spark-BigQuery connector (load jobs — the same real-BQ path Beam already takes).
- `Counters` / `require_balance` → end-of-batch `groupBy().count()` → `assert` → `run_ledger` row. (The engine already asserts at batch close, not mid-shuffle, so this is faithful.)
- `record_lineage` → the not-migrated DataFrame → BigQuery.
- I4-Python: Spark native SQL supports parameter binding, so this port fixes the f-string SQL interpolation at source** (see "Relationship to open review findings" below).

Acceptance

All 8 criteria pass unchanged.

Trade-offs

Pros: the most familiar batch engine for mainframe-data teams; Dataproc Serverless is pay-per-batch; mature BigQuery connector; the four dispositions are very legible as column filters; scales to the stated "nn billion" trivially; **the two hardest invariants — the shared `route_intake` and the content-based `survivor_rank` — port with zero logic change.**

Cons: batch-only; dedup is a full shuffle (fine for batch, not for streaming); mid-pipeline balance becomes end-of-batch balance (acceptable, since the engine already asserts at batch close).

Effort: M.

C1: unchanged.

Biggest risk: the local-emulator path. Dataproc Serverless has no local equivalent, so `make run-initial` needs `--master local[*]` or keeps the Beam / DirectRunner path for dev — splitting the "one pipeline shape, two runners" property the project currently enjoys.

Proposal C — dbt + BigQuery + thin Cloud Run ingest (warehouse-native, SQL-first)

The framing

BigQuery is named in C1. Apache Beam is **not** in C1 — it is an implementation detail inside the boundary. Moving the engine *into* BigQuery is the most C1-faithful option of the four: it removes a runtime C1 never mentioned and doubles down on one it does.

Swaps

Beam pipelines → **dbt models** on BigQuery (staging → intermediate → marts). One thin **Cloud Run Job** ingests raw TDS into BigQuery (fixed-width → raw string → BQ `LOAD`). Dataform SQLX folds into dbt. `json_producer` batch-emit and the loader stay as small Cloud Run services; recon stays Java (reads BQ). Orchestrator → Composer calling dbt, or Cloud Workflows.

The dedup-as-SQL pattern `QUALIFY ROW_NUMBER() OVER (PARTITION BY dedup_key ORDER BY survivor_rank) = 1` is **already used by `json_producer` for batching** — this proposal generalises that existing pattern to the whole engine.

Engine port map

Stage	SQL / tool
parse	BigQuery <code>LOAD</code> the raw line as <code>STRING</code> + a staging view with <code>SUBSTR</code> / <code>SPLIT</code> extracts driven by the mapping YAML offsets (the copybook-offset logic the harness already proves yields identical fields)
filter(exclude)	<code>WHERE</code> on <code>client_type</code>
dedup	<code>QUALIFY ROW_NUMBER() OVER (PARTITION BY dedup_key ORDER BY survivor_rank) = 1</code> (the json_producer pattern, generalised)
map	<code>JOIN</code> s + <code>CASE</code> for enum mapping; the six reason codes surface here
schema	a Cloud Run post-hook calling the same <code>jsonschema.Draft202012Validator</code> (keeps criterion 4 byte-identical — full Draft 2020-12 isn't native SQL)
the four dispositions	four CTEs / models → four BigQuery tables (same table names → <code>acceptance.py</code> unchanged)
Counters / require_balance	a dbt <code>test (assert src_read = migrated + not_migrated)</code> + a post-hook <code>run_ledger</code> row
record_lineage	a dbt model unioning the three not-migrated CTEs
contracts/	now consumed by a SQL generator , not imported by Python engine code
I4-Python	fixed at source (dbt parameter binding)

Acceptance

All 8 criteria pass unchanged; dbt tests add a **second** acceptance layer on top of the harness oracle.

Trade-offs

Pros: fewest moving parts; the warehouse *is* the engine — balance becomes a SQL assertion over lineage, the fastest thing to audit; cheapest to operate; no Dataflow shuffle; most C1-faithful (BigQuery is in C1, Beam is not).

Cons: fixed-width parse in SQL is ugly (raw-string + `SUBSTR`); the JSON-Schema stage needs a Cloud Run post-hook; **loses "engine as shared importable code across Python + Java"** — `contracts/` becomes SQL-generator input, so drift-prevention shifts from "one imported module" to "the generator is the single source"; the Reason enum becomes a `CHECK` constraint (weaker than a Python enum).

Effort: M-L (the contract-driven SQL generator is the hard part).

C1: unchanged — and the most C1-faithful of the four.

Biggest risk: the SQL generator must reproduce `survivor_rank` 's content-only canonical-JSON tie-break **byte-for-byte**, or the dedup survivor differs from the Beam run on reordered input. Mitigated by a golden-test that runs the same harness fixture through both runtimes and asserts identical survivors.

Proposal D — Cloud Run Jobs + Cloud Workflows (lean MVP, no

managed batch engine)

The insight

The repo *already proves this shape locally*. `local/scripts/run_pipeline.py` literally shells out to each stage in sequence (`python -m pipelines.file_processor.pipeline , ...`), and the pipeline mains already have a CLI form. This proposal productionises that onto Cloud Run + Workflows — the local mirror is already its prototype.

Swaps

Beam pipelines → the existing modules as **containerised Cloud Run Jobs** (mains already CLI-form; `run_pipeline.py` already invokes them). Dataflow → nothing (in-process per job). Composer → **Cloud Workflows** (or Eventarc). Dataform stays. The Java apps → Cloud Run Jobs. BigQuery, GCS, Kafka and Terraform stay.

At scale, in-process dedup per job will not hold → push dedup to the BigQuery side via `QUALIFY ROW_NUMBER()` (the `json_producer` pattern, again).

Engine port map

Each stage runs as a Cloud Run Job writing the four dispositions to BigQuery; `Counters / require_balance` → in-job aggregate → `run_ledger` row, assert before exit 0; `record_lineage` → BigQuery, written by the file-processor job; dedup at scale → BigQuery-side `QUALIFY` (moved out of job memory into SQL). The existing `Dockerfile.toolbox / Dockerfile.javaapp` containerise both sides directly.

Acceptance

All 8 criteria pass unchanged.

Trade-offs

Pros: simplest operational model; cheapest at small / medium scale; no Dataflow or Composer cost; each stage is a plain container; closest to the existing `run_pipeline.py` shape.

Cons: the scale ceiling. A Cloud Run Job has an 8-hour max plus CPU / memory limits, so at "nn billion" a single job per stage will not scale → manual sharding, at which point you have reimplemented a poor man's Spark. Loses distributed shuffle; batch-only.

Effort: S-M.

C1: unchanged.

Biggest risk: the scale ceiling — fine for the 405-record prototype and medium loads, but it does not scale to the stated "nn billion" without sharding.

Proposal E — Streaming / CDC variant ⚠ C1 MODIFIED (one input edge)

This is the one proposal that changes C1. It is flagged as a variant, not a drop-in same-C1 swap.

Swaps

Mainframe TDS-file-over-SFTP input → **Db2 CDC via Datastream** → **Pub/Sub**; a streaming engine (Beam Streaming on Dataflow) consumes Pub/Sub → BigQuery + Target System. The `.FLG` / `.CHS` batch artefact contract → per-window manifests **or** dropped in favour of a continuous ledger. `run_id` / window → watermark + periodic snapshots. `json_producer` batch-write → direct Kafka / Pub/Sub streaming produce (Kafka is already in C2).

The C1 modification (explicit)

```
- MF -->|"TDS fixed-width files + .FLG/.CHS artefacts [PGP over SFTP/GCS]"| SYS
+ MF -->|"Db2 change stream (INSERT/UPDATE/DELETE) [Datastream → Pub/Sub]"| SYS
```

One input edge changes; the other six external systems, the five actors, and the central responsibility stay. This is faithful to C1 *intent* (two-door engine, balancing proof, record-level evidence) but revises the C1 *input contract*. It requires the Migration Engineer and the Auditor to re-sign C1.

Engine port map

Per-event `parse` → `filter` → `map` → `schema` in a `ProcessFunction`; `dedup` → keyed state **with TTL** (`survivor_rank` within a window — streaming dedup is harder: state cleanup, late events); the four dispositions → four side outputs → four BigQuery tables (streaming inserts / file-loads per checkpoint); `require_balance` → **per-checkpoint** assertion; `run_ledger` row per checkpoint; `record_lineage` per event. The shared `route_intake` / `survivor_rank` port unchanged.

Harness / acceptance (the honest cost)

Needs a CDC-emitting harness variant (`generate.py --format cdc` materialising the same 405 records as change events + the same manifest). Criteria 2, 3, 4, 6, 7, 10 pass essentially unchanged; 1 and 9 pass with reinterpretation (per-window rather than per-batch); **criteria 5 (batches of 200) and 8 (the five batch artefacts) break and require acceptance + `artefacts.json` revision**. That is the honest cost of this variant.

2026-08-21 note. The criterion numbers above are against the old 10-criterion set; the simplification drops old #2 (excluded-exact) and #10 (delta) and rennumbers to 8 — read this breakdown as the old numbering.

Trade-offs

Pros: real-time migration + continuous reconciliation; delta runs become the default; Kafka is already in C2.

Cons: changes C1; the mainframe team may not offer Db2 CDC; streaming dedup / state is the hardest ops problem in the set; highest complexity; the harness needs a streaming counterpart.

Effort: L.

C1: modified — one input edge.

Biggest risk: the mainframe may not expose Db2 CDC — and if it cannot, this proposal is not available at all. Feasibility is gated on an external party.

Comparison matrix

	B. Spark / Dataproc	C. dbt + BigQuery	D. Cloud Run + Workflows	E. Streaming CDC
C1	unchanged	unchanged (most faithful)	unchanged	modified (1 edge)
Swaps	Beam → Spark	Beam → SQL in BigQuery	Beam → Cloud Run Jobs	file/SFTP → CDC/Pub/Sub
Orchestrator	Composer	Composer / Workflows	Workflows	Composer
Streaming-capable	no	no (incremental dbt)	no	yes (native)
Scale	highest (batch)	high (BigQuery)	capped (8h/job)	high
Ops weight	low (serverless)	lowest	low	highest
Local / GCP duality	yes (local vs serverless)	yes (BQ emulator vs BQ)	yes (run_pipeline.py already)	no
Fixes I4-Python at source	yes	yes	no	no
Effort	M	M-L	S-M	L
Repo intent	—	—	(local mirror exists)	—

Recommendation

- **Lowest-effort, lowest-ops, batch-scale** → **B (Spark on Dataproc Serverless)**. Near-literal DataFrame translation; Spark parameter binding fixes I4-Python at source.
- **Most C1-faithful + cheapest to operate, SQL-strong team** → **C (dbt + BigQuery)**. Removes a runtime C1 never names (Beam), doubles down on one it does (BigQuery); dbt tests add a second acceptance layer. Make-or-break is fixed-width parsing in SQL.
- **Lean MVP on GCP without Dataflow** → **D (Cloud Run + Workflows)**. The repo already proves the shape locally via `run_pipeline.py`; the scale ceiling is the catch.
- **Real-time + CDC available** → **E (Streaming / CDC)**. But it modifies C1 and is the highest-effort, highest-risk option.

A pragmatic sequence: **D as the MVP** → **B or C as the production batch runtime** → **E only if CDC is available**. All of B-D satisfy the same C1, the same `contracts/`, the same harness oracle, and the same 8 acceptance criteria unchanged.

Relationship to the open review findings

These proposals were developed alongside a re-review of the current implementation. Two items from that review interact with the proposals above and are worth recording here so the document is self-contained:

1. **HIGH — project-id / region hardcoded in tracked Terraform** (`terraform/envs/dev/main.tf:32` inside the non-overridable `backend "gcs"` block, `:79` and `:84` as defaults; `git ls-files` confirms it is committed; the repo is public; violates the repo's own `README.md:429-431` rule). This is an IaC / repo-hygiene defect of the *current* implementation. It is **orthogonal** to every proposal here — each proposal keeps the Terraform layer and inherits the same fix: move project / region to a gitignored `terraform.tfvars`, and template the tfstate backend bucket name at `terraform init` time or via a per-env wrapper.
2. **MEDIUM (mitigated, not fixed at source) — I4 Python SQL interpolation**. `run_id` is f-string-interpolated into SQL at ~10 sites in the current Python engine (including `DELETE` s at `pipelines/file_processor/pipeline.py :402` and `:498`). It is **doubly-guarded** — `run_id` passes through `require_identifier("--run-id", ...)` (`^[A-Za-z0-9_.-]+$`) at every CLI entrypoint, so no user-supplied string can become injection — but it is not fixed at source. Two of the proposals above (**B** Spark, **C** dbt) fix this at source via native parameter binding; that is noted in each proposal's port map and in the comparison matrix.

Everything else from the re-review (the `Config.validate()` bypass, the stale DAG comment, the `BigQueryRest` pagination gap, the per-app workload-identity service accounts, the delta criterion-10 scoping, the I4-Java path) is already **fixed** in the working tree at the time of writing and is not affected by these proposals.